

Data Structures and Algorithms I.

University of Szeged – Robert Nagy

2025/26. Spring Semester

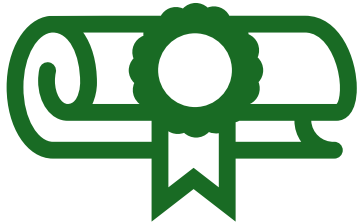
Introduction to the course



Requirements



2 In-class, 45-minute-long Written Assignments (50 + 50 points)



Potential extra points for class work (up to 10)

Required Minimum

15 points per W.A.

50/100 in total



Attendance IS mandatory!

About the teacher

- 3rd year student (Undergraduate)
- Software Developer Intern (1 year of experience as of February 2026)



I don’t know everything!



…But maybe I can teach you something new!

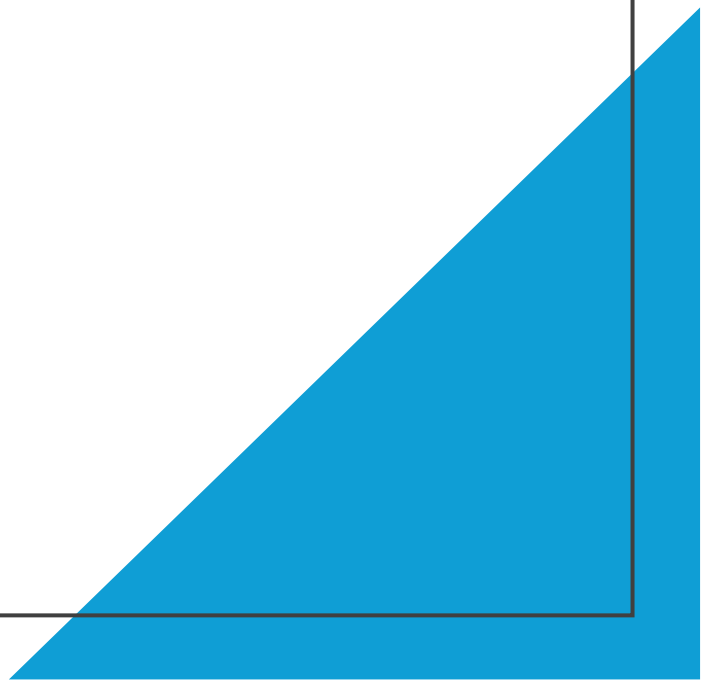
Motivation

Solving real world problems with computers



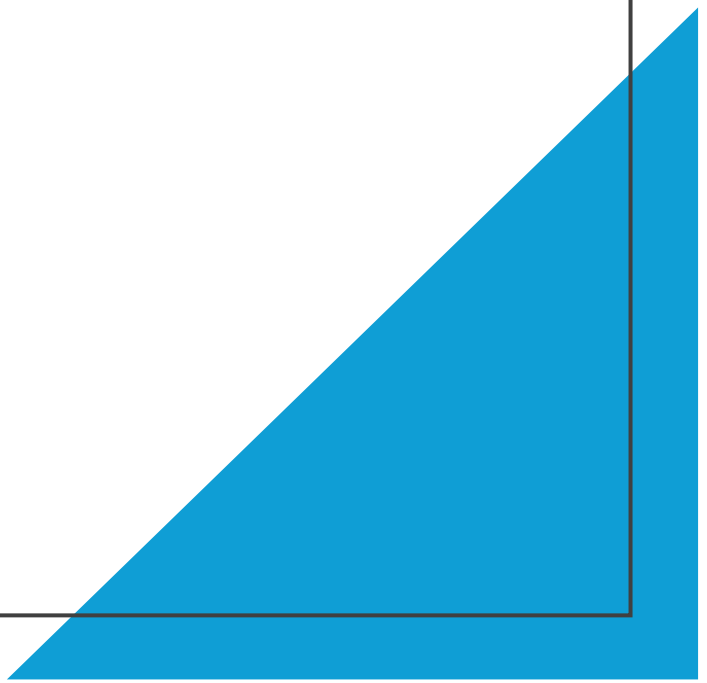
Foundations

What is an algorithm?



Cost of computation

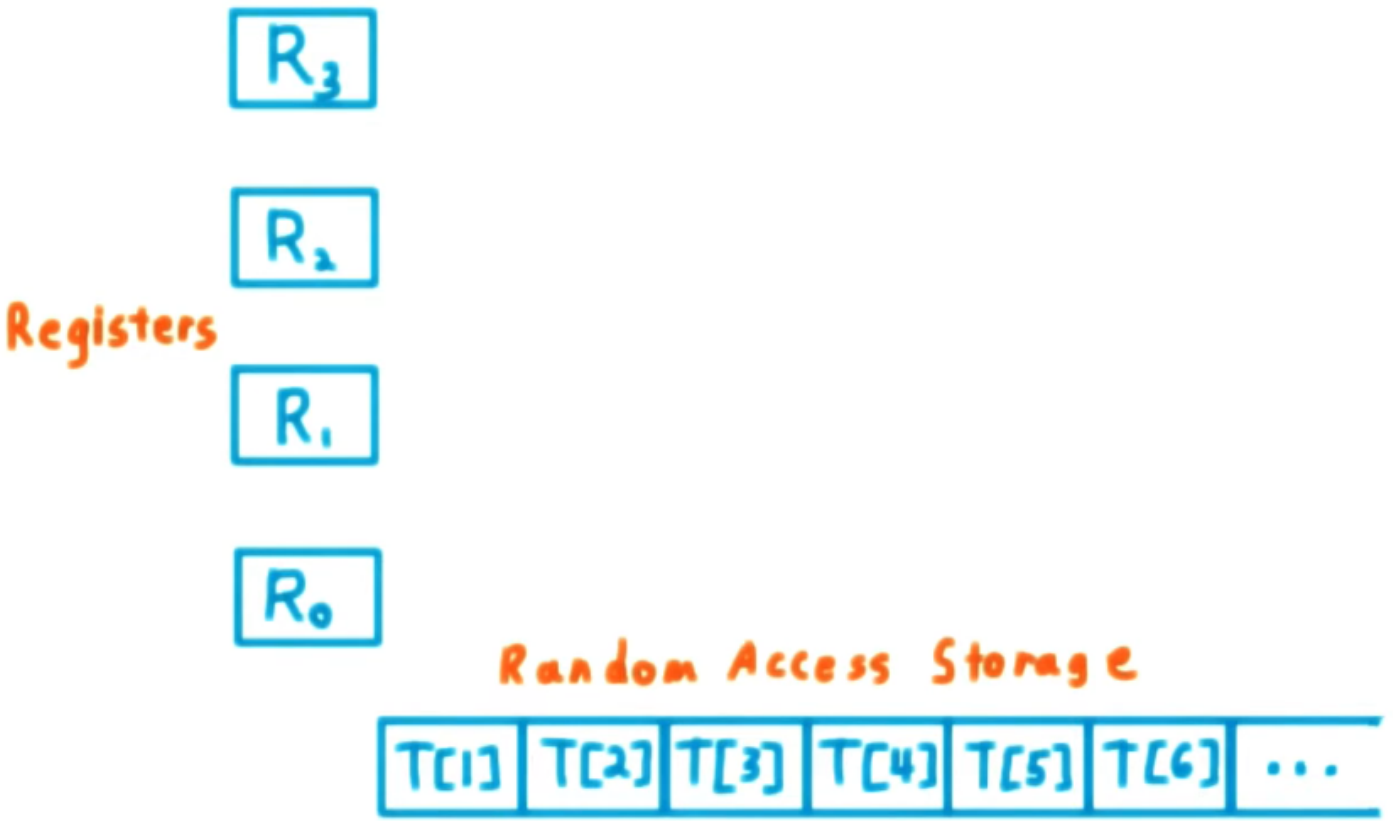
- Time?
- Space?



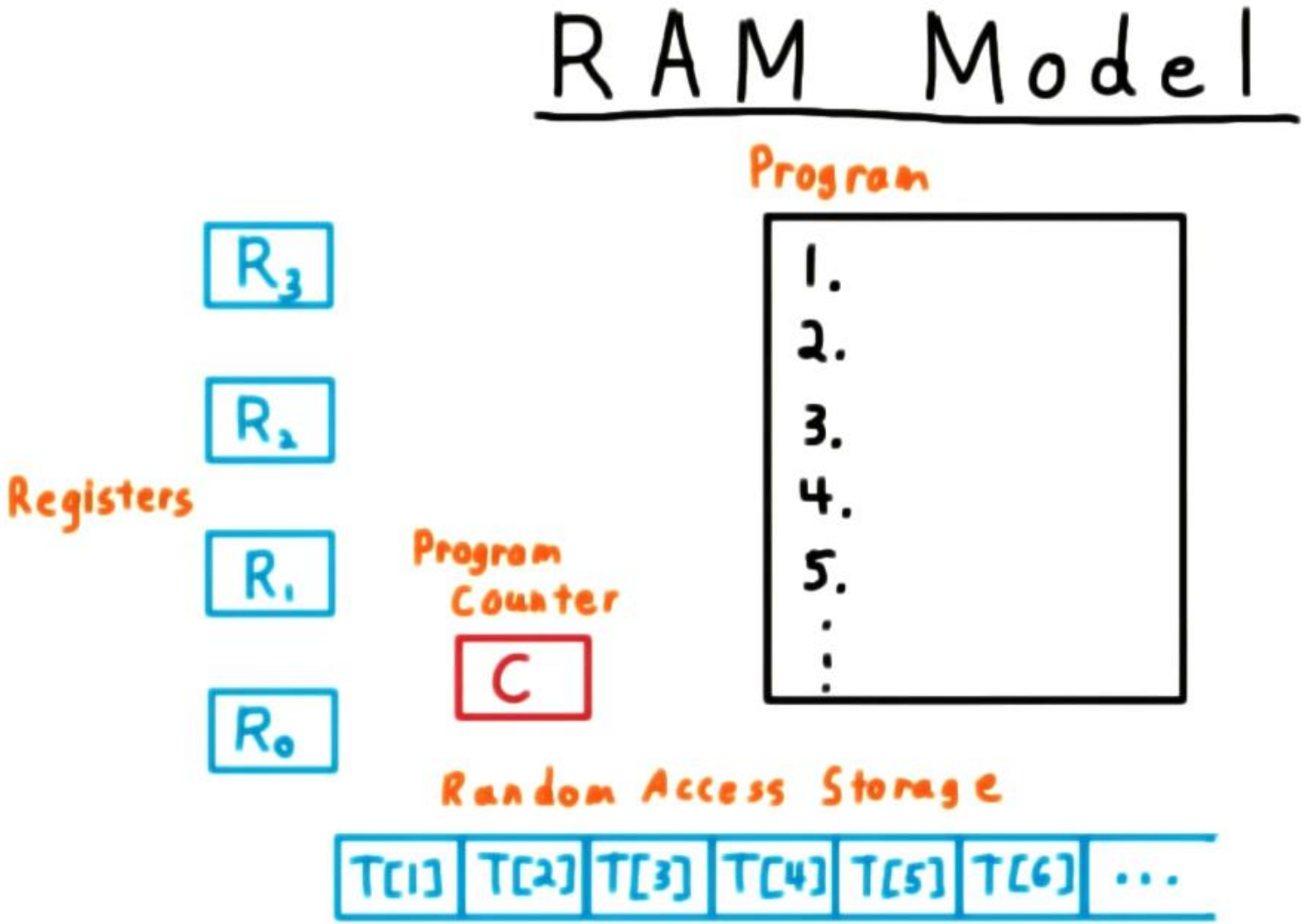
Model of computation

- RAM Model (Random Access Machine)

RAM Model

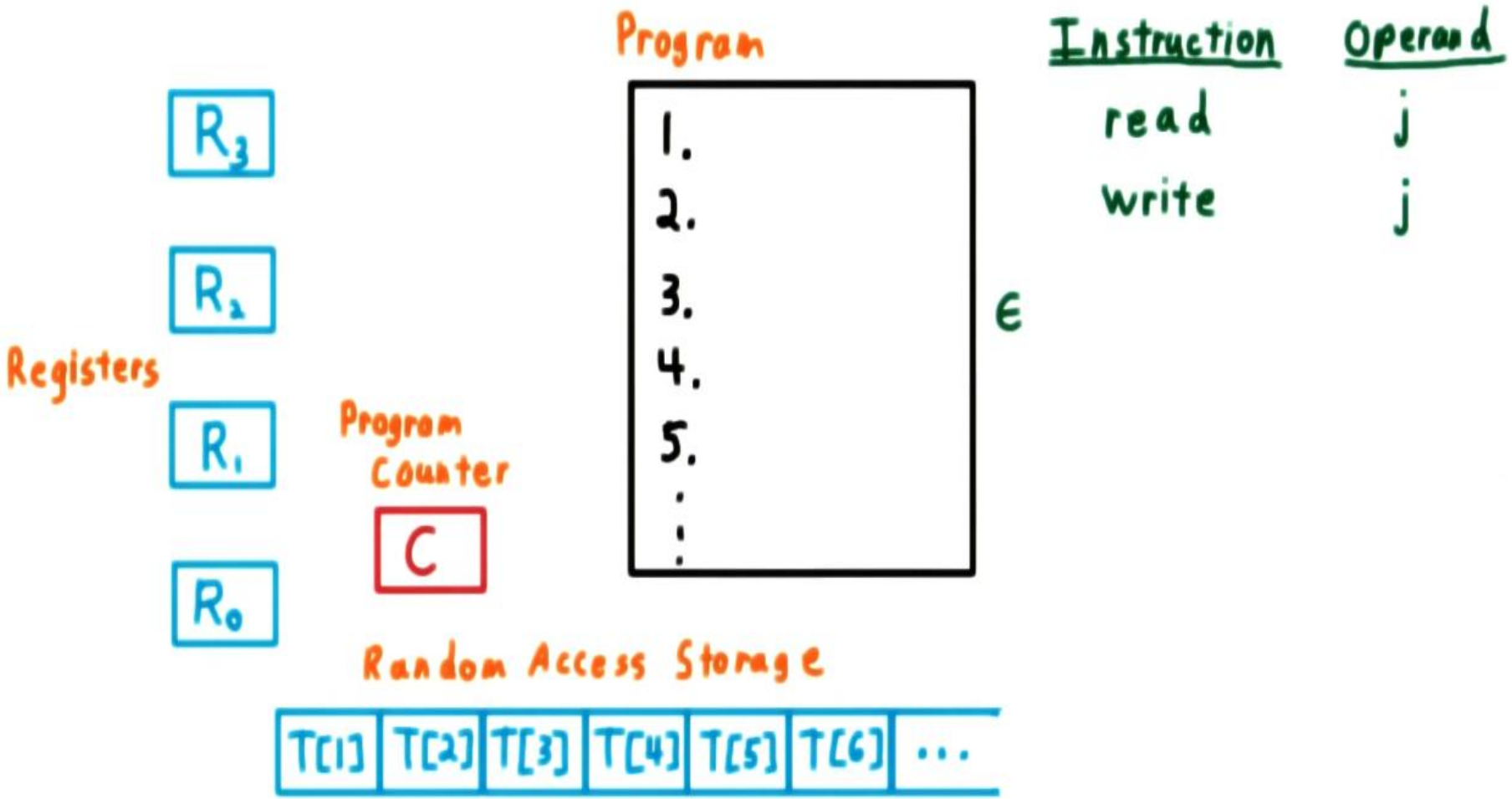


Source: https://www.youtube.com/watch?v=wkBjD0Fvl6U

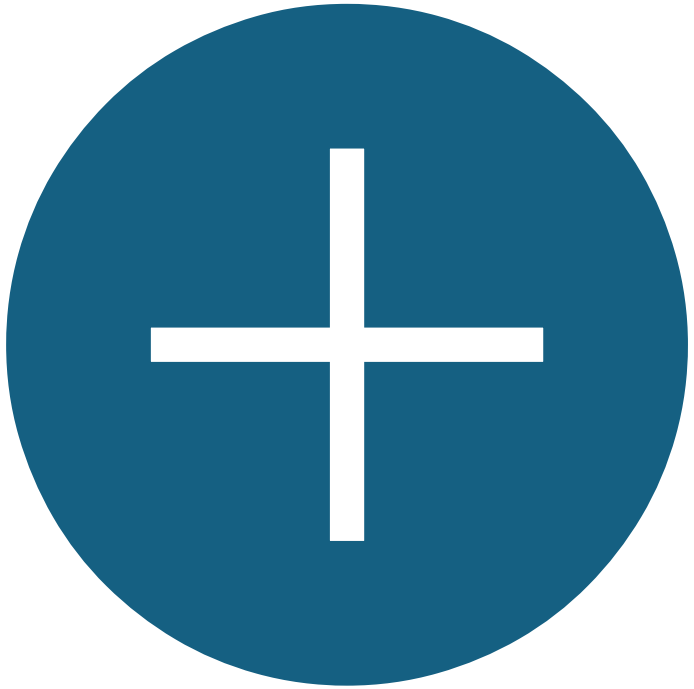


Source: https://www.youtube.com/watch?v=wkBjD0Fvl6U

RAM Model



Source: https://www.youtube.com/watch?v=wkBjD0Fvl6U



Basic operations

Constant time!

- Arithmetic (+, -, *, /, %...)
- Data movement (Load, Store...)
- Control (Method call, conditional jump, etc...)

Example: Insertion Sort



Source: https://www.youtube.com/watch?v=Q1JdRUh1_98

Insertion sort - Pseudocode

INSERTION-SORT(A, n)	*cost*	*times*

Insertion sort - Pseudocode

INSERTION-SORT(A, n)	*cost*	*times*
1 **for** $i = 2$ **to** n	c_1	n
2 $\quad key = A[i]$	c_2	$n-1$

Insertion sort - Pseudocode

INSERTION-SORT(A, n)	*cost*	*times*
1 **for** $i = 2$ **to** n	c_1	n
2 $key = A[i]$	c_2	$n-1$
3 // Insert $A[i]$ into the sorted subarray $A[1:i-1]$.	0	$n-1$
4 $j = i-1$	c_4	$n-1$

Insertion sort - Pseudocode

INSERTION-SORT(A, n)	*cost*	*times*
1 **for** $i = 2$ **to** n	c_1	n
2 $key = A[i]$	c_2	$n-1$
3 // Insert $A[i]$ into the sorted subarray $A[1:i-1]$.	0	$n-1$
4 $j = i-1$	c_4	$n-1$
5 **while** $j > 0$ and $A[j] > key$	c_5	$\sum_{i=2}^{n} t_i$
6 $A[j+1] = A[j]$	c_6	$\sum_{i=2}^{n} (t_i - 1)$

Insertion sort - Pseudocode

INSERTION-SORT(A, n)	*cost*	*times*
1 **for** $i = 2$ **to** n	c_1	n
2 $key = A[i]$	c_2	$n-1$
3 // Insert $A[i]$ into the sorted subarray $A[1:i-1]$.	0	$n-1$
4 $j = i-1$	c_4	$n-1$
5 **while** $j > 0$ and $A[j] > key$	c_5	$\sum_{i=2}^{n} t_i$
6 $A[j+1] = A[j]$	c_6	$\sum_{i=2}^{n} (t_i - 1)$
7 $j = j-1$	c_7	$\sum_{i=2}^{n} (t_i - 1)$
8 $A[j+1] = key$	c_8	$n-1$

Can we **formalize** the “*time it takes to solve a problem*” as a **function** of the “*size of the input*”?

The total time

- Let the total time our algorithm consumes be *T(n)*



$$T(n) =$$

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1)$$

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{i=2}^{n} t_i + c_6 \sum_{i=2}^{n} (t_i - 1)$$
$$+ c_7 \sum_{i=2}^{n} (t_i - 1) + c_8(n-1) \; .$$

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{i=2}^{n} t_i + c_6 \sum_{i=2}^{n} (t_i - 1)$$
$$+ c_7 \sum_{i=2}^{n} (t_i - 1) + c_8(n-1) \; .$$

This looks complicated…

Let’s simplify things!

edge cases

Worst case

Average case

Best case

Best case

$$
\begin{aligned}
T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\
&= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) \; .
\end{aligned}
$$

Best case

$$
\begin{aligned}
T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\
&= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) \; .
\end{aligned}
$$

The input array is already sorted. Therefore, $t_i = 1$

$$
\begin{aligned}
T(n) = {} & c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{i=2}^{n} t_i + c_6 \sum_{i=2}^{n} (t_i - 1) \\
& + c_7 \sum_{i=2}^{n} (t_i - 1) + c_8(n-1) \; .
\end{aligned}
$$

Best case

$$
\begin{aligned}
T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\
&= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) \ . \\
&= a\,n + b
\end{aligned}
$$

Best case

$$
\begin{aligned}
T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\
&= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) \ . \\
&= a\,n + b
\end{aligned}
$$

This is a Linear Function of *n*.

Worst case

- The input array is in reverse order
- Therefore, $t_i = i \ \ for\ i = 2, 3, \ldots, n$

Worst case

$$\sum_{i=2}^{n} i = \left(\sum_{i=1}^{n} i \right) - 1$$

Worst case

$$\sum_{i=2}^{n} i = \left(\sum_{i=1}^{n} i \right) - 1$$

$$= \frac{n(n+1)}{2} - 1$$

Worst case

$$\sum_{i=2}^{n}(i-1) = \sum_{i=1}^{n-1} i$$

$$= \frac{n(n-1)}{2}$$

Worst case

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right)$$
$$+ c_6\left(\frac{n(n-1)}{2}\right) + c_7\left(\frac{n(n-1)}{2}\right) + c_8(n-1)$$

Worst case

$$
\begin{aligned}
T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right) \\
&\quad + c_6\left(\frac{n(n-1)}{2}\right) + c_7\left(\frac{n(n-1)}{2}\right) + c_8(n-1) \\
&= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8\right) n \\
&\quad - (c_2 + c_4 + c_5 + c_8) \, .
\end{aligned}
$$

Worst case

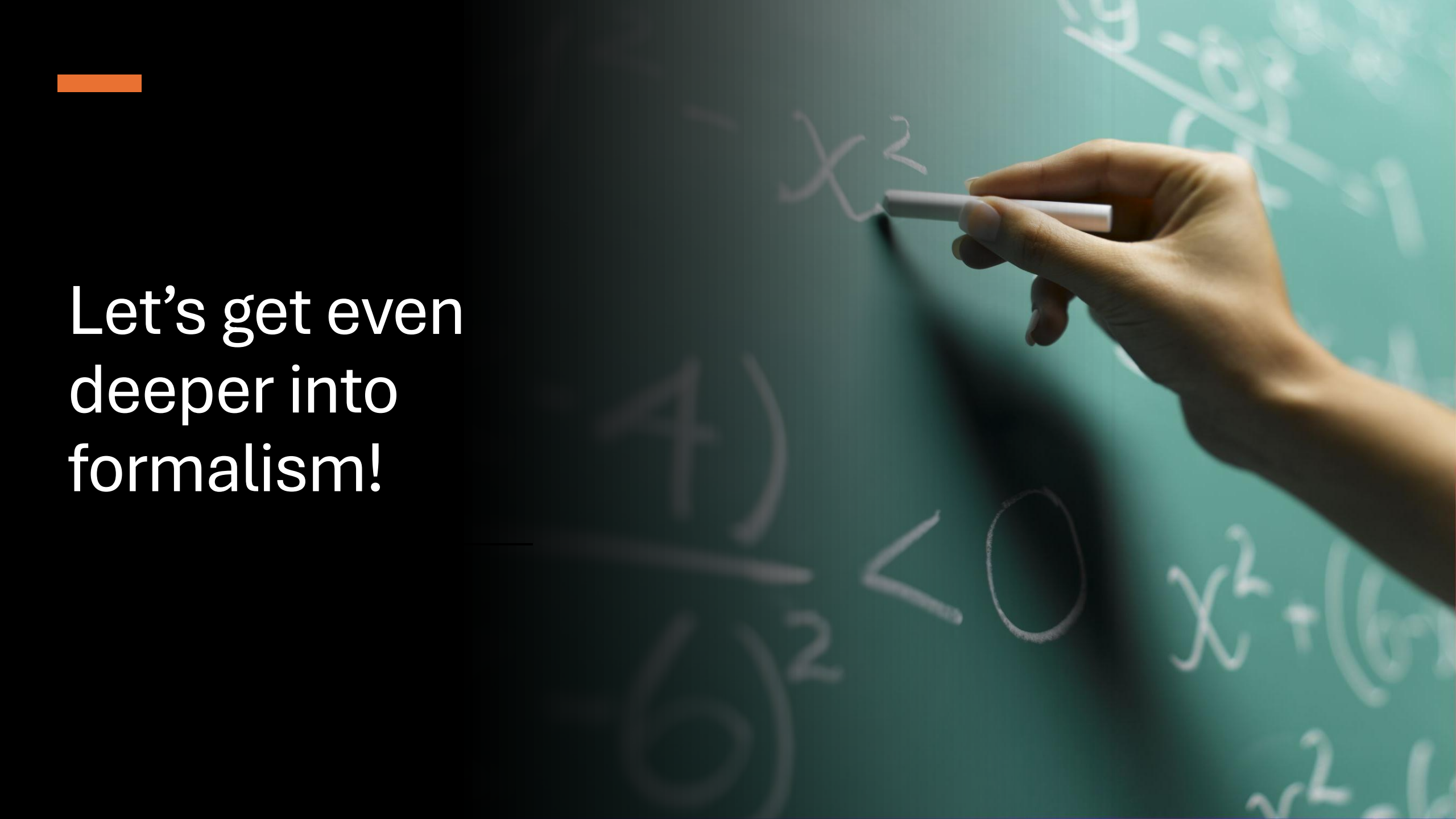
$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right)$$
$$+ c_6\left(\frac{n(n-1)}{2}\right) + c_7\left(\frac{n(n-1)}{2}\right) + c_8(n-1)$$
$$= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8\right) n$$
$$- (c_2 + c_4 + c_5 + c_8) .$$

$$= an^2 + bn + c$$

The worst case is a quadratic function of *n*.

$$T(n) = an^2 + bn + c$$

Let’s get even deeper into formalism!

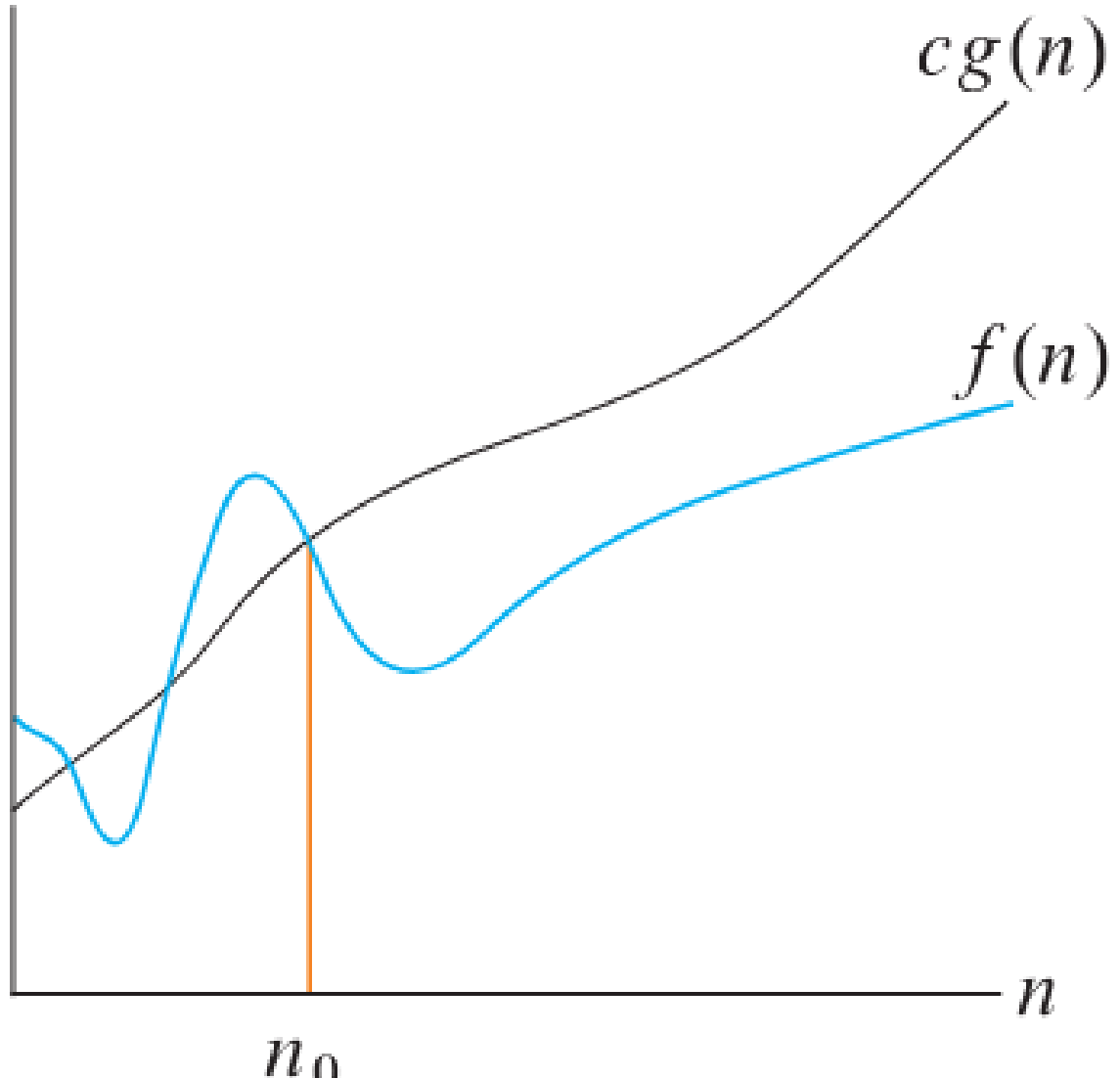


Time complexity

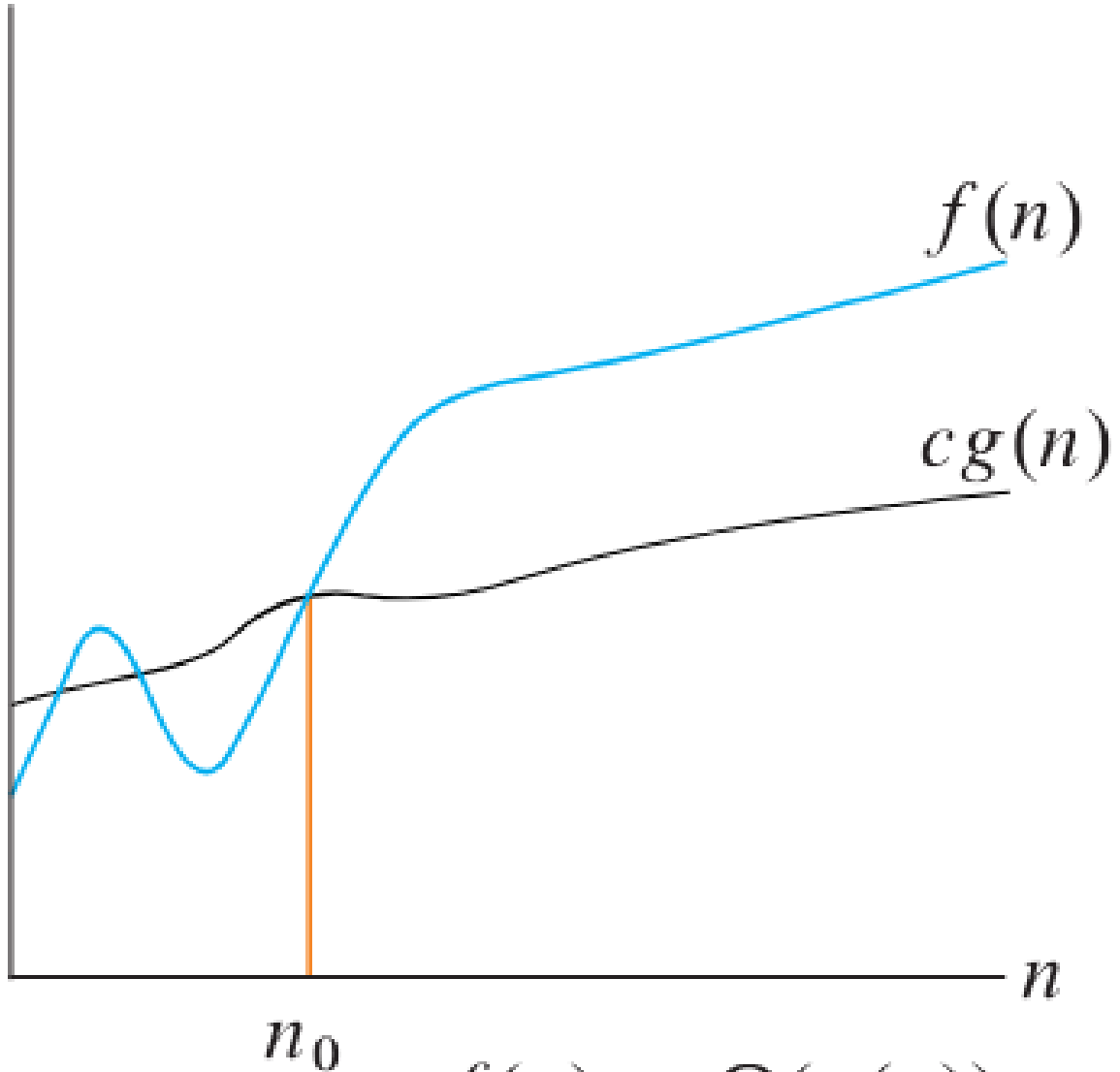
Upper bound - O

Lower bound - Ω

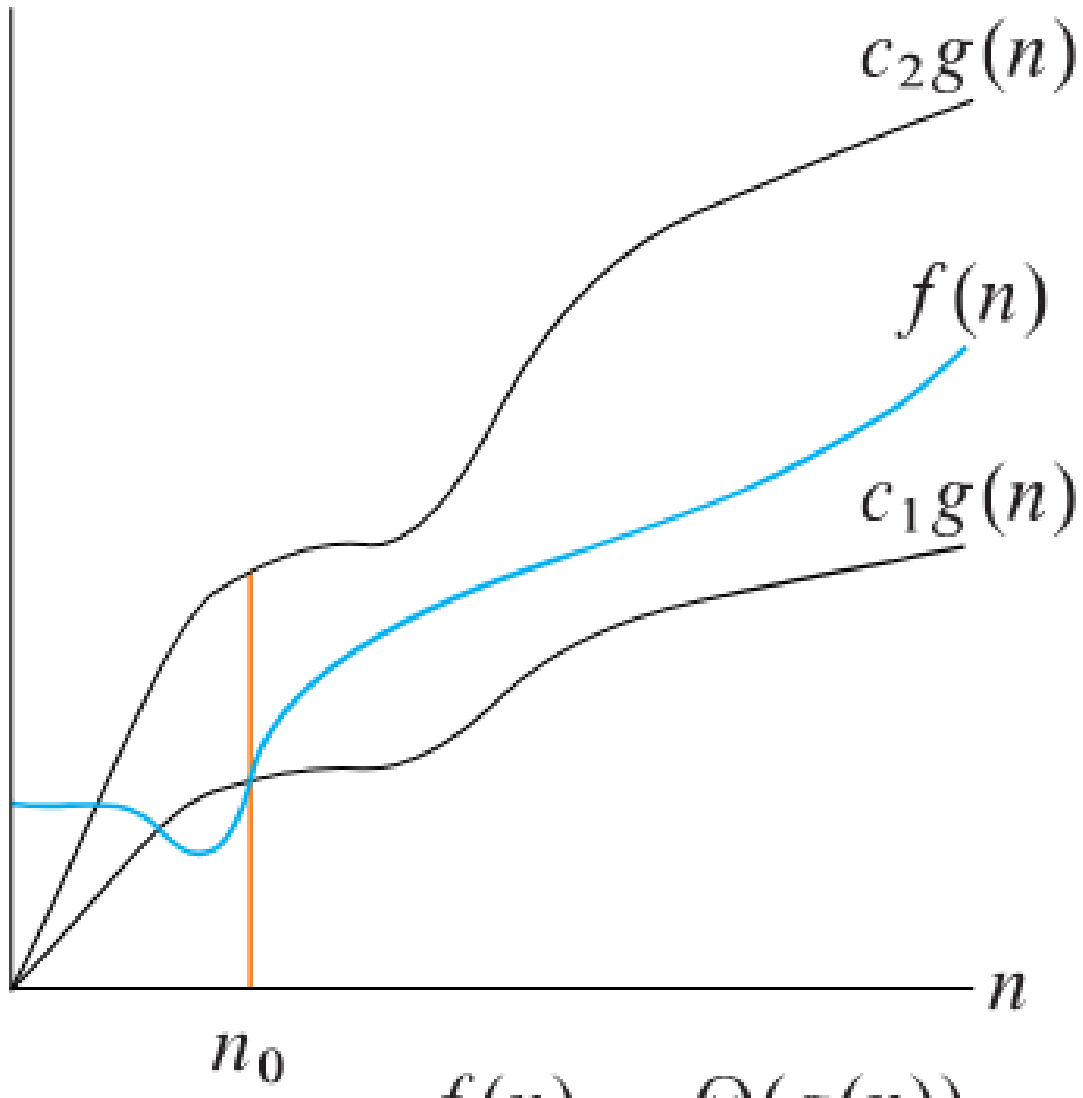
Tight bound - Θ



$$f(n) = O(g(n))$$



$$f(n) = \Omega(g(n))$$



$$f(n) = \Theta(g(n))$$

So, we can say:

- $Best\ case = \Omega(n)$
- $Worst\ case = O(n^2)$

So, we can say:

- $Best\ case = \Omega(n)$
- $Worst\ case = O(n^2)$

…What about the average case?

On average

$$t_i = \frac{i}{2}$$

On average

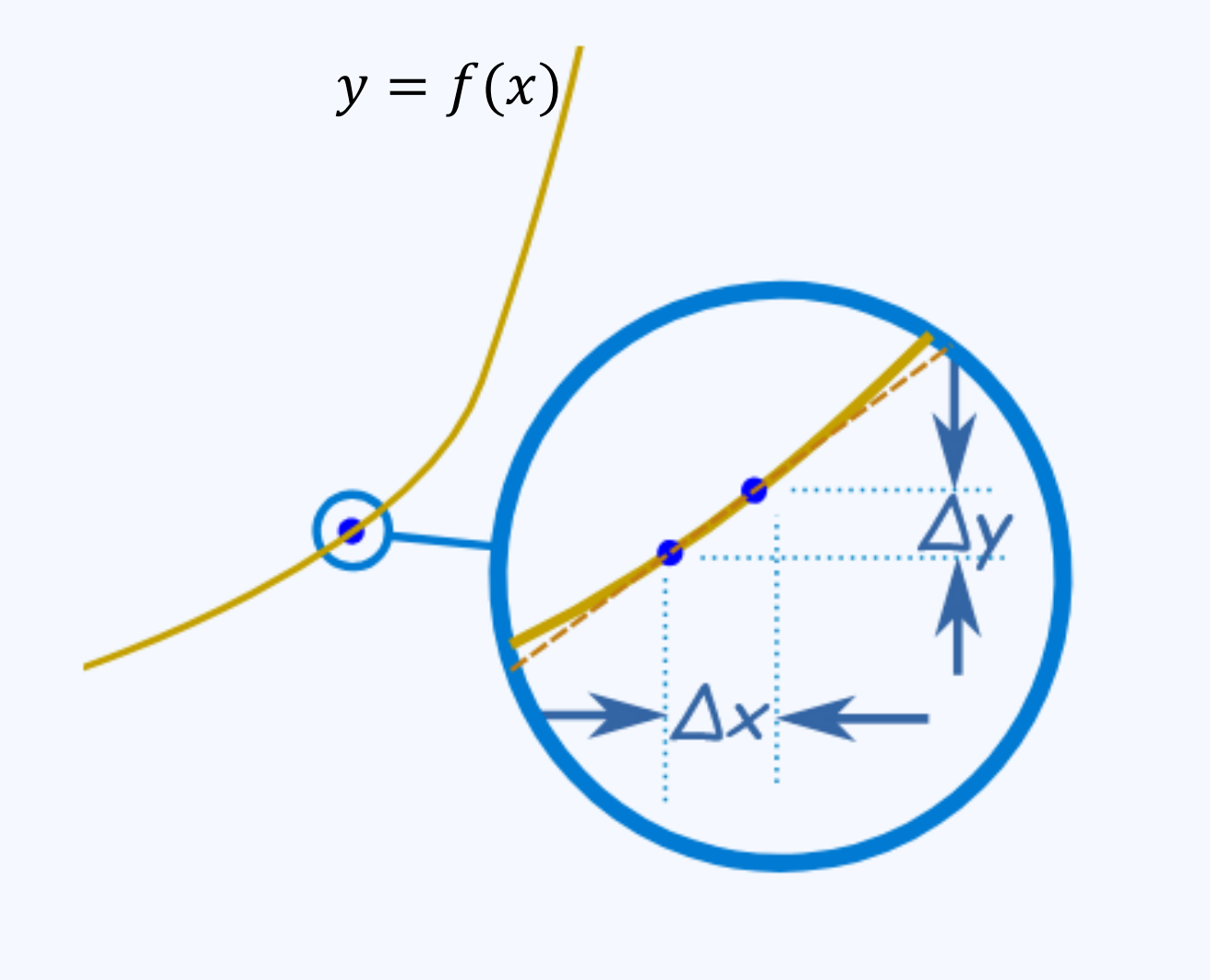
$$\sum_{i=1}^{n-1}(t_i) = \sum_{i=1}^{n-1}\left(\frac{i}{2}\right) = \frac{1}{2}\sum_{i=1}^{n-1}(i) =$$

$$= \frac{1}{2} * \frac{n(n-1)}{2} = \frac{1}{4}n(n-1)$$

The average case is often roughly as bad as the worst case.

What really matters – The ***Rate*** of ***growth***

Sounds familiar?



https://www.mathsisfun.com/definitions/differentiation.html

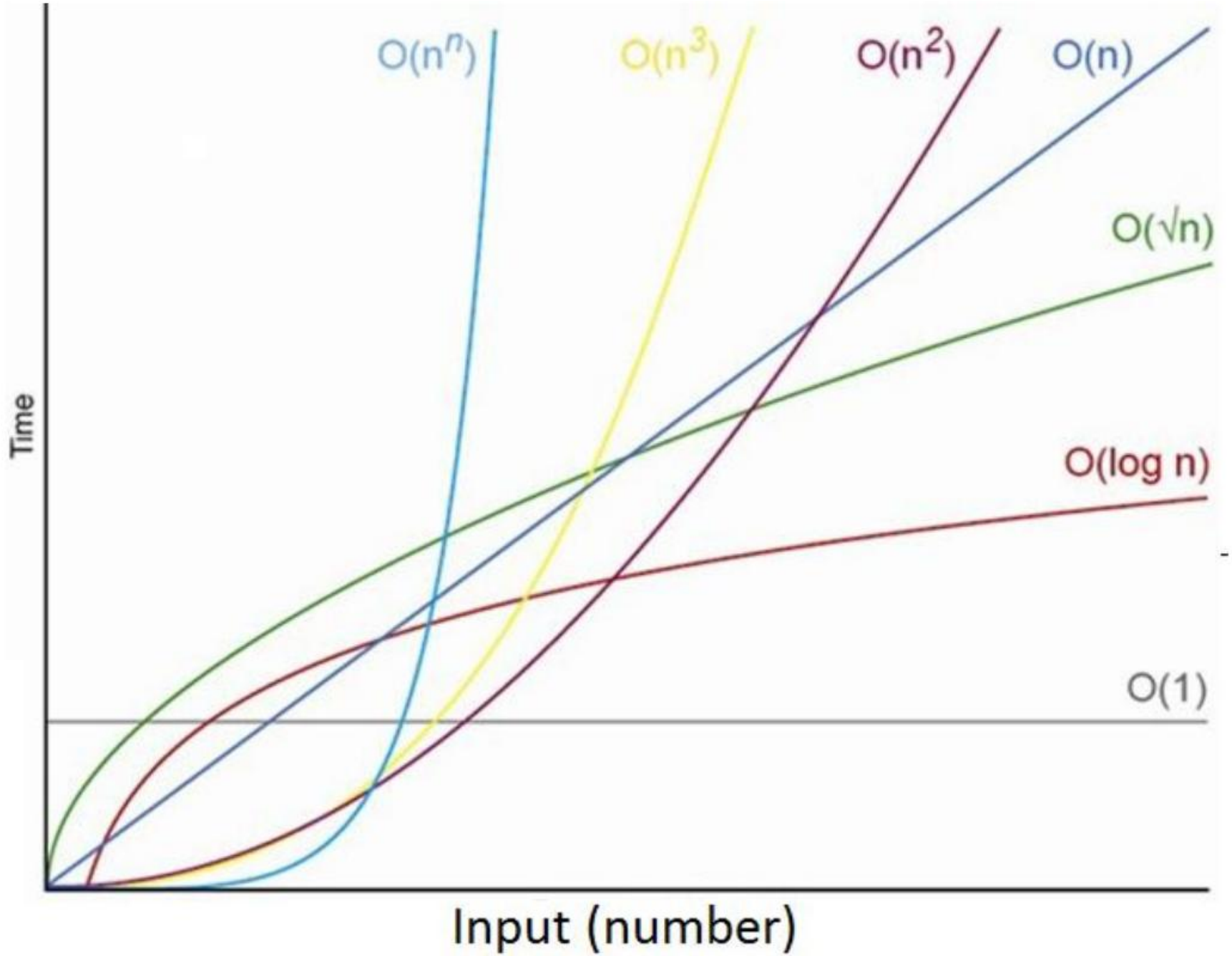
Exercises

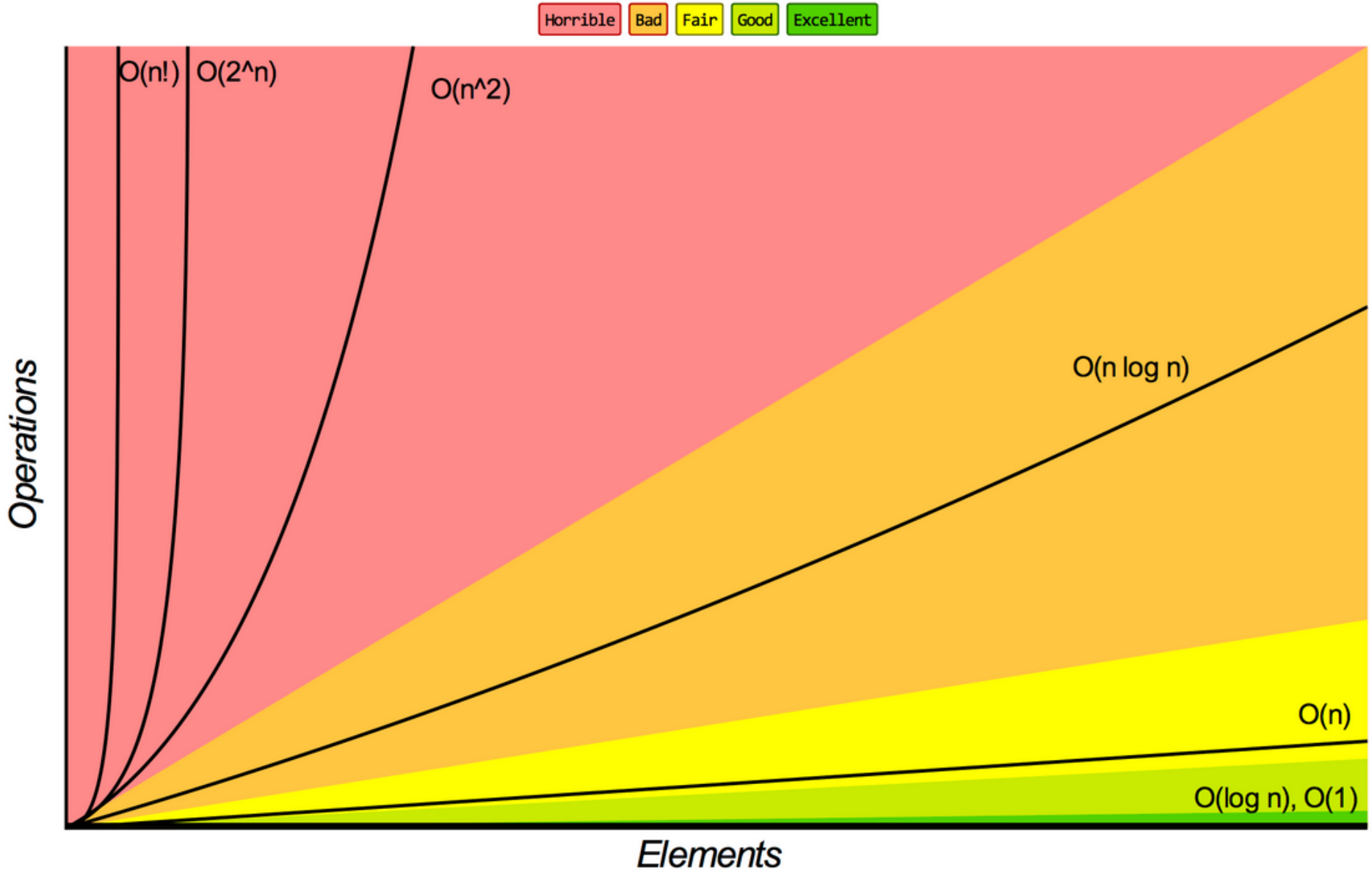
Determine the upper bound for the following functions:

1. $T(n) = 3n + n^2 + \ln(n)$
2. $T(n) = \ln(n) * n + n^{0.5}$
3. $T(n) = 1 + 2^n + \ln(\ln(n))$

Determine which function grows faster

1. $f(n) = n^2 + 4n$ --- OR --- $g(n) = n * ln(n) + 13$
2. $h(n) = 4^{log_2(n)} + \frac{3}{2}n$ --- OR --- $j(n) = 6n + 7 * \sqrt[3]{log_2(n)}$
3. $k(n) = n^{log_2(3)} + \sqrt[2]{n} + n^3$ --- OR --- $l(n) = n^{log_2(7)} + n^2$


O(n^n)
O(n^3)
O(n^2)
O(n)
O(√n)
O(log n)
O(1)
Time
Input (number)



Source: **bigocheatsheet.com**

$$O(N^N) > O(N!) > O(2^N) > O(N^2) > O(N \log N) > O(N) > O(\log N) > O(1)$$

Quiz time!

- https://read.learnyard.com/dsa/quiz-time-complexity/
- https://www.101computing.net/big-o-notation-quiz/